



FabLab

Université Paul Sabatier

Projet structure de données - L2 CUPGE UPSSITECH 2025/2026

TOUNKARA Aissata | Travail individuel

Présentation du projet



Objectif : Développer un programme en langage C pour gérer les ressources d'un Fablab (PCs, machines, formations, outils...) avec gestion des prêts et des utilisateurs.

Personnes

Demandeurs et prêteurs avec connexion par email + mot de passe chiffré

Ressources

Physiques ou immatérielles, avec dépendances entre elles

Prêts

Historique complet : emprunts, retours, transferts, périodes



Architecture du projet



fab.h

Structures de données + prototypes de toutes les fonctions

fab.c

Fonctions utilitaires : vider_buffer, lire_chaine

personnes.c

CRUD complet des personnes (ajouter, modifier, supprimer, afficher)

ressources.c

Gestion des ressources physiques et immatérielles

prets.c

Emprunts, retours, historique, transferts

menus.c

Interface : menu connexion, admin, utilisateur

crypto.c

Chiffrement des mots de passe

sauvegarde.c

Chargement, sauvegarde ligne par ligne, export JSON

main.c

Point d'entrée : init, chargement, menu, sauvegarde, libération

Structures de données



struct Personne

- id, nom, prenom
- mail, telephone
- mdp_hache (chiffré)
- est_actif (suppression logique)

struct Pret

- id_personne (emprunteur)
- id_ressource (ressource empruntée)
- date_debut, date_fin
- est_termine

struct Ressource

- id, nom, type, description
- est_physique, peut_etre_partagee
- id_ressource_mere (-1 si aucune)
- est_active

struct Fablab

- *personnes, nb, cap, next_id
- *ressources, nb, cap, next_id
- *prets, nb, cap
- mdp_admin (chiffré)

Mode d'emploi



Compilation & lancement

```
$ gcc -o fab2 *.c
$ ./fab2
ou sur Windows :
$ fab2.exe
Sur linux:
$ make
  ./fab2.exe
```

Menu de connexion

```
1 - Connexion Administrateur
2 - Connexion Utilisateur
6 - Quitter
MDP admin par défaut : admin
```

- 1 Lancer le programme — les données sont chargées automatiquement depuis donnees.txt
- 2 Se connecter en Admin (mdp : admin) ou Utilisateur (email + mot de passe)
- 3 Admin : gérer les personnes, changer le mot de passe, exporter en JSON
- 4 Utilisateur : emprunter/rendre des ressources, consulter ses prêts, modifier ses infos
- 5 Quitter — les données sont sauvegardées automatiquement dans donnees.txt

Choix techniques



Suppression logique (est_actif = 0)

Préserve l'intégrité des prêts historiques. Supprimer physiquement déplacerait les indices et casserait les références entre prêts et personnes.

Tableau dynamique (malloc + realloc)

Capacité doublée automatiquement quand le tableau est plein. Évite une limite fixe arbitraire et économise la mémoire au démarrage.

Sauvegarde texte ligne par ligne

Plus lisible que le binaire (fwrite). On peut ouvrir donnees.txt dans un éditeur et vérifier les données. Format portable entre OS.

Chiffrement par somme ASCII pondérée

Solution simple et explicable en L2, sans bibliothèque externe. Les mots de passe ne sont jamais stockés en clair.

4 structures (Personne, Ressource, Pret, Fablab)

Pret séparé car c'est un événement dans le temps avec sa propre existence — indépendant de la personne et de la ressource.

Export JSON standard

Format universel lisible par n'importe quel outil externe. Respecte la contrainte du sujet sans bibliothèque JSON externe.

Difficultés rencontrées



Choix des structures

Comprendre comment découper le problème : pourquoi une structure Pret séparée ? Après réflexion : un prêt est un événement dans le temps avec sa propre existence.

Gestion du buffer (`vider_buffer`)

Mélange `scanf` + `fgets` : le `\n` laissé par `scanf` était lu par `fgets`. Solution : appeler `vider_buffer()` systématiquement entre les deux.

Installation de l'environnement

Tentative avec VirtualBox + Ubuntu échouée (affichage figé). Retour sur VS Code avec GCC Windows. Au départ Code::Blocks était utilisé mais le Makefile y est difficile à configurer.

Format JSON

Concept inconnu au départ. Implémenté manuellement avec `fprintf` sans bibliothèque externe — apprentissage du format par la pratique.

Bilan individuel



Ce que je savais déjà

- ✓ Projets C/C++ (supermarché, université)
- ✓ Structures de données simples (1-2 max)
- ✓ Gestion de fichiers avec fwrite/fread
- ✓ Gestion du buffer (projet environnement)
- ✓ Code::Blocks comme environnement principal

Ce que j'ai appris

- ★ Makefile et compilation multi-fichiers
- ★ VS Code comme environnement de dev
- ★ Git et GitHub (versioning du code)
- ★ Format JSON et export manuel
- ★ Chiffrement simple des mots de passe
- ★ Projet C avec 4+ structures imbriquées



Conclusion

Merci pour votre attention !

